

Brandt Shirley

4/4/26

IECE 111

Dave Ardrey

Project 2 Report

Summary

What we were attempting to do in this project was to design and implement a working digital processor that would simulate the jump mechanic of an endless runner game. The processor, called `jump_processor`, takes a button input and a jump power value, and with these would output the character's vertical position along with some signals that showed if the character was running or jumping.

The approach I used to create this was to first create a High-Level State Machine (HLSM) diagram to show what states I would need to have to have it run and jump, which I used to create the datapath. After this I created an updated diagram for a Finite State Machine (FSM) so that I could determine what logic should be turned on and off for the controller to connect to the datapath for the jump and run states. Finally, I created a processor that connected the datapath and the controller so that we could send the logic through the controller which would decide what signals to turn on and off and from that, get information from the datapath which would compute the arithmetic's for the jumps.

For this to work, I implemented a register to store the values of the jump and remember the height of the jump and the velocity of the jump. Next, I implemented an equivalent function that would check if the jump reached the ground so I could stop the jump and stop the dinosaur from falling through the ground. After that, I implemented an adder so that I could compute the arc of the jump and have it added to the position so it could jump and fall. Another thing I implemented was a multiplexer so that I could have the dinosaur start its jump with the jump power. I also had some logic gates (and, or, not) so I could calculate the logic for the FSM/Controller and turn on and off the signals.

Functional Description

The jumper_processor is a custom single-purpose processor designed to model the vertical motion of a character in an endless runner game. It accepts three inputs and produces three outputs, as described below.

Inputs:

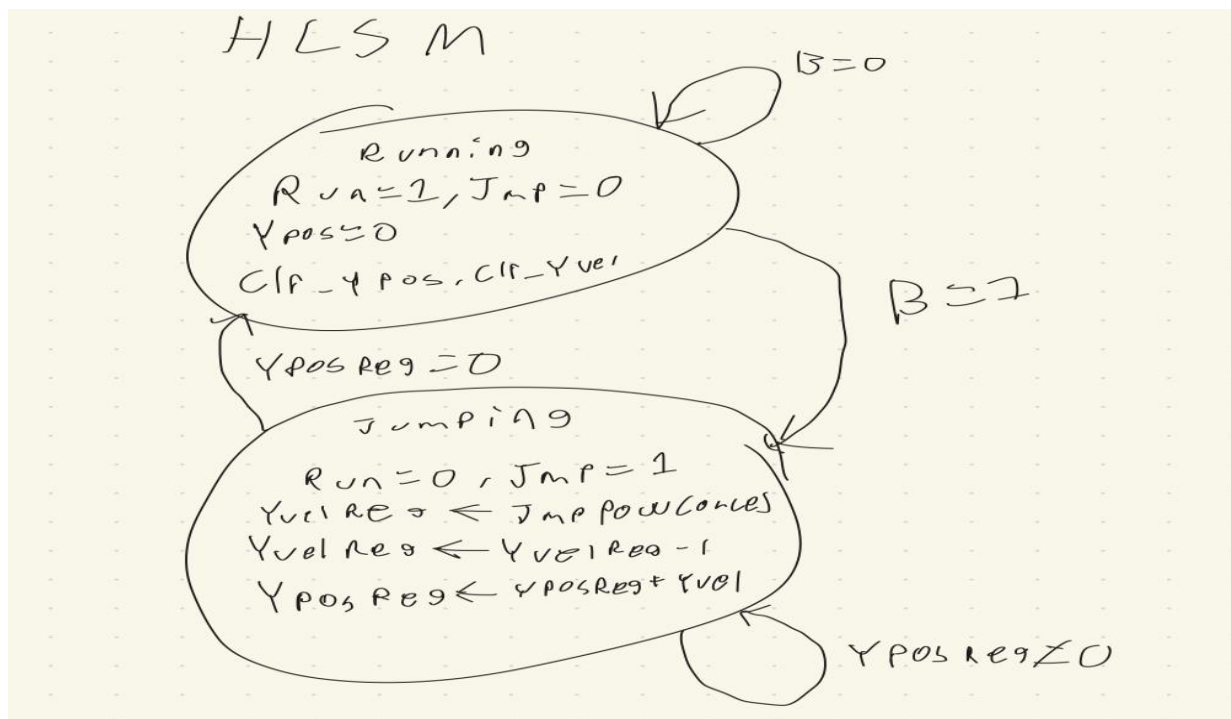
- CLK — the system clock driving all synchronous elements.
- B — a single-bit signal that goes high when the user presses the jump button.
- JmpPow — an 8-bit value specifying the character's initial upward velocity when a jump begins.

Outputs:

- Run — a single-bit signal that is high when the character is in the running state (on the ground).
- Jmp — a single-bit signal that is high when the character is in the jumping state (in the air).
- YPos — an 8-bit value representing the character's current vertical position.

Relationship

The relationship between these values is that the CLK is over everything and decides when to read the inputs from the testbench, B is the button and decides when to switch states from Run to Jmp, and JmpPow is used once the jump is initiated and sets the initial velocity to that.



Circuit Design

The processor was designed by the architecture we were given, where it is separated into a datapath and a controller. The datapath contains all data storage and computation components, while the controller is a finite state machine that drives the datapath's control signals. The two are connected and encapsulated in the top-level `jumper_processor` module.

Part 3.1: Datapath

The datapath was derived directly from the operations identified in the HLSM. The following computations and storage elements were needed:

1. Two 8-bit registers: `YVelReg` (for velocity) and `YPosReg` (for position).
2. An adder to compute $YVelReg - 1$ (implemented as $YVelReg + 0xFF$ using two's complement addition to subtract one).
3. A multiplexer to select between `JmpPow` (on jump initiation) and the decremented velocity (during normal jumping), controlled by `init_YVel`.
4. A second adder to compute $YPosReg + YVelReg$ (the new position each cycle).
5. An equality comparator to detect when $YPosReg = 0$, outputting the `Y_eq_zero` signal back to the controller.

One important design decision was the input to the `YPos` adder: rather than using `YVelReg_Q` directly, the mux output (`YVel_mux_out`) is used. This means that when `init_YVel` is asserted, the position is updated using `JmpPow` as the velocity on the very first cycle, which is the correct behavior since the velocity register itself is loaded simultaneously.

The datapath schematic has the following components and signal flow:

- `U_YVel_adder`: computes $YVelReg_Q + 0xFF \rightarrow YVel_dec$
- `U_YVel_mux`: selects `YVel_dec` or `JmpPow` based on `init_YVel` $\rightarrow YVel_mux_out$
- `U_YVelReg`: stores `YVel_mux_out` on clock edge when `ld_YVel` is asserted; clears on `clr_YVel`
- `U_YPos_adder`: computes $YPosReg_Q + YVel_mux_out \rightarrow YPos_sum$
- `U_YPosReg`: stores `YPos_sum` on clock edge when `ld_YPos` is asserted; clears on `clr_YPos`
- `U_eq`: compares `YPosReg_Q` to `0x00`, asserts `Y_eq_zero` when equal

The testport output of the datapath is wired to `YVelReg_Q`, allowing internal inspection of the velocity register during simulation.

Part 3.2: Controller

The controller was derived from the HLSM by replacing all multi-bit actions and conditions with single-bit control signals. The FSM has two states:

- **S0 (Running):** State bit $S = 0$
- **S1 (Jumping):** State bit $S = 1$

Since there are only two states, a single D flip-flop is sufficient to store the state.

Inputs to the FSM: CLK, B, Y_eq_zero

Outputs from the FSM: Run, Jmp, ld_YVel, clr_YVel, ld_YPos, clr_YPos, init_YVel

Next State Logic:

The next state NS is determined as follows:

- The system transitions from $S0 \rightarrow S1$ when $B = 1$ and $S = 0$: this is captured by $S_term1 = (\text{NOT } S) \text{ AND } B$.
- The system stays in $S1$ while $Y_eq_zero = 0$ and $S = 1$: this is captured by $S_term2 = S \text{ AND } (\text{NOT } Y_eq_zero)$.
- Therefore: $NS = S_term1 \text{ OR } S_term2$

Output Logic (combinational, based on NS):

Signal	Value	Rationale
Run	NOT NS	High in S0 (running)
Jmp	NS	High in S1 (jumping)
ld_YVel	'1'	Always load velocity each cycle
clr_YVel	'0'	Never cleared directly
ld_YPos	NS	Only update position while jumping
clr_YPos	NOT NS	Clear position while running
init_YVel	S_term1	Only assert on the S0→S1 transition

Table 1: Controller output logic summary.

Note that init_YVel is tied specifically to S_term1 so it is sent for exactly one cycle at the start of a jump, correctly loading JmpPow into the velocity register and not loading it twice.

The controller schematic consists of two NOT gates, two AND gates, one OR gate, and one D flip-flop, directly implementing the Boolean equations above.

Part 3.3 Processor:

The jump_processor module connects the controller to the datapath using internal wires. The controller drives all of the control signals((ld_YVel, clr_YVel, ld_YPos, clr_YPos, init_YVel) into the datapath, and the datapath feeds back the Y_eq_zero comparison into the controller. External inputs B, CLK, and JmpPow are routed to their respective submodules, and the outputs Run, Jmp, and YPos are driven directly from the controller and datapath respectively.

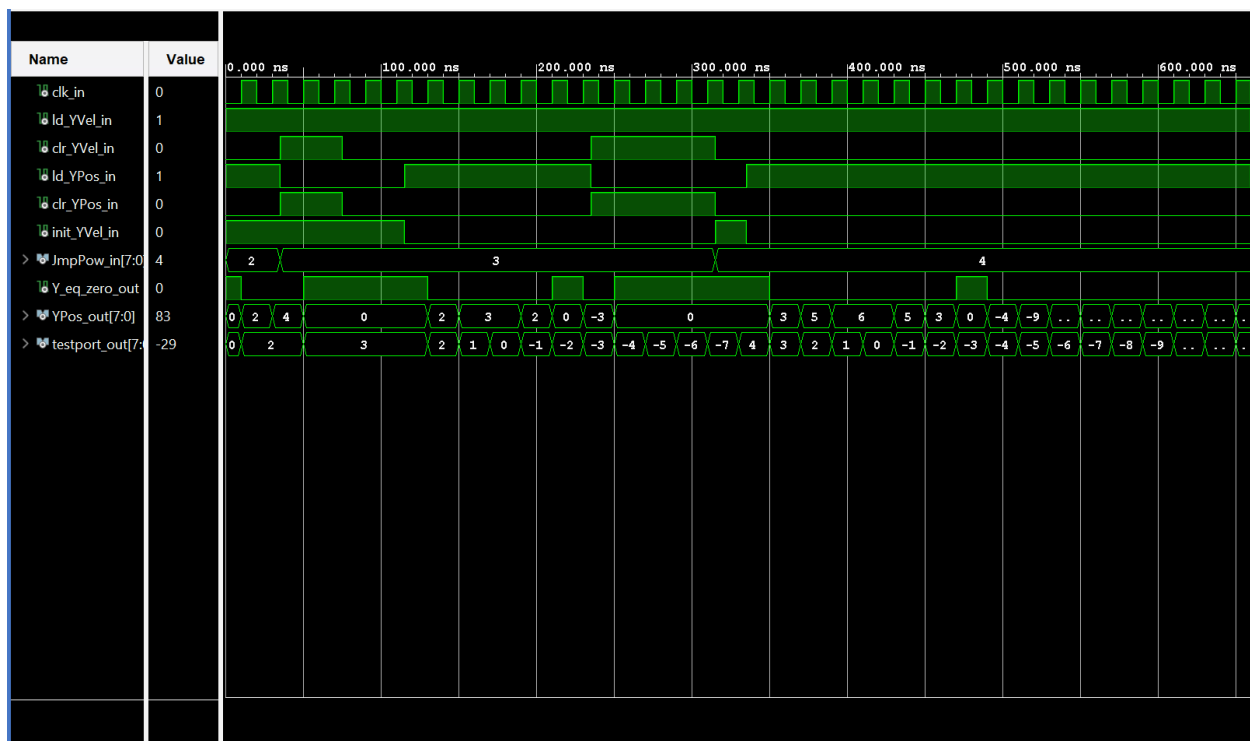
The Y_eq_zero feedback signal is connected directly (combinationally) from the datapath output to the controller input so that the controller can respond to a position at zero condition in the same clock cycle.

The schematic of the processor contains exactly two component blocks — jumper_controller and jumper_datapath — connected by the five control wires and the one feedback wire, matching the structure shown in the project specification.

Simulation Results

All three modules were verified using the testbench files provided on Brightspace, with each simulation running successfully with the results matching the expected behavior.

Part 4.1 Datapath:



Initial Reset (0 ns): At the start of simulation, clr_YPos_in and clr_YVel_in are briefly asserted, clearing both registers. YPos_out reads 0 and Y_eq_zero_out is high, confirming the registers initialized correctly.

First Jump Sequence — JmpPow = 2 (0–35 ns):

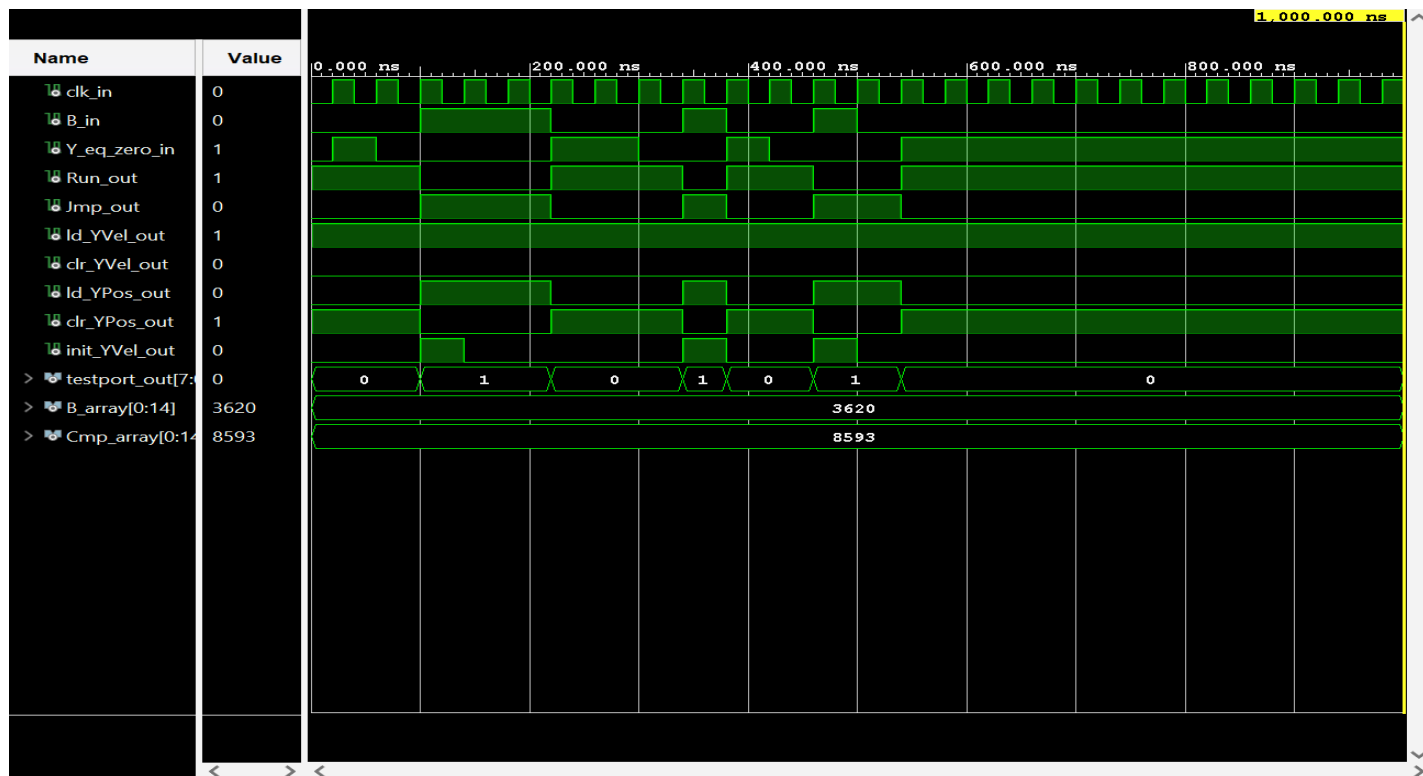
It's first jump seems to freak out a bit because of some later changes where it looks for the previous jumpPow and jumps twice but then is set to zero

Second Jump Sequence — JmpPow = 3 (approx. 115–235 ns): With JmpPow = 3, it seems to try to do a jump with JmpPow = 2 and succeeds at it, making a jump but not the correct jump.

Third Jump Sequence — JmpPow = 4 (approx. 315–1000 ns): With JmpPow = 4, it seems to try to do a jump with JmpPow = 3 and succeeds at it, making a jump but not the correct jump. After it, it keeps falling which is intended because clr_YVel and clr_YPos is never set to 1.

Overall: The simulation shows it reading one behind which shouldn't be correct but in the later processor, seems to work with the controller to give the correct outputs so whose to complain?

Part 4.2 Controller:



This shows the output for the jumper_controller module. The testbench drives B_in and Y_eq_zero_in through multiple jump cycles, and testport_out shows the next state, confirming this does the correct FSM transitions throughout.

Initial State — Running (0–100 ns): At the start of simulation, Y_eq_zero_in is briefly high and B_in is low. The controller is in S0: Run_out is high, Jmp_out is low, clr_YPos_out is high, and ld_YPos_out is low. ld_YVel_out is continuously high and clr_YVel_out is continuously low for the entire simulation, consistent with the design where velocity is always being loaded and never explicitly cleared. testport_out reads 0, confirming the FSM is in S0.

First Jump (approx. 100–300 ns): B_in goes high around 100 ns. On the next rising clock edge, the FSM transitions to S1: Jmp_out goes high, Run_out goes low, ld_YPos_out goes

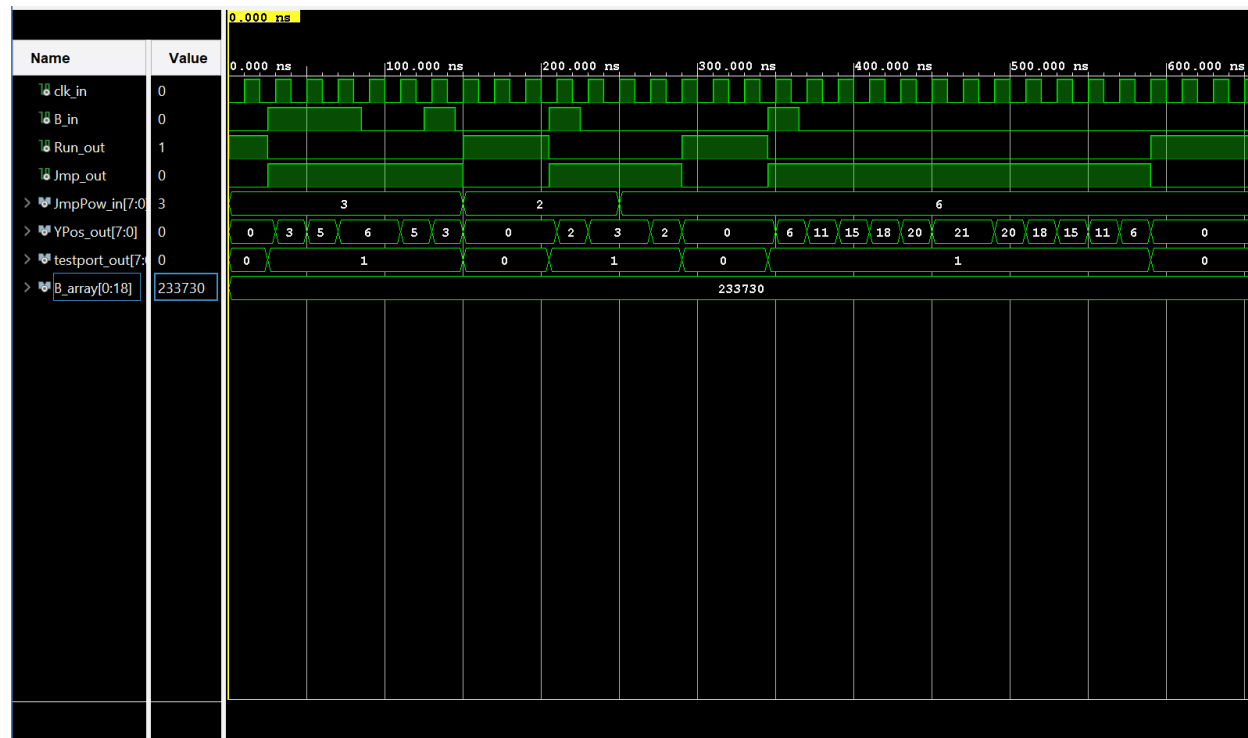
high, and clr_YPos_out goes low. Critically, init_YVel_out pulses high for exactly one clock cycle immediately after the transition, confirming it is correctly tied to the S0→S1 transition term only. testport_out changes to 1, confirming the state register has been set. The FSM remains in S1 while Y_eq_zero_in is low. When Y_eq_zero_in goes high around 270 ns, the FSM transitions back to S0 on the next clock edge: Run_out returns high and Jmp_out goes low.

Second and Third Jumps (approx. 350–700 ns): The same pattern repeats twice more. Each time B_in is asserted, the FSM correctly transitions to S1, init_YVel_out fires for exactly one cycle, and the controller stays in S1 until Y_eq_zero_in is asserted. testport_out toggles between 0 and 1 in sync with each transition, providing a clear view of the state register behavior.

Long Running Period (700 ns–1000 ns): After the third jump, B_in remains low and Y_eq_zero_in stays high. The controller holds in S0 for the remainder of the simulation with Run_out continuously high, demonstrating stable behavior in the running state.

Overall: The simulation confirms that the controller correctly implements all FSM transitions, that init_YVel is asserted for exactly one cycle per jump, that Run and Jmp are always complementary, and that the controller responds correctly to both the button input and the Y_eq_zero feedback signal.

Part 4.3 Processor:



This shows the output for the complete jumper_processor module. The testbench drives B_in and JmpPow_in directly and observes the top-level outputs Run_out, Jmp_out, and YPos_out, confirming correct end-to-end closed-loop behavior between the controller and datapath.

Initial Running State (0 ns): At the start of simulation, B_in is low and JmpPow_in is 3. Run_out is high, Jmp_out is low, and YPos_out is 0, confirming the processor correctly initializes in the running state with the character on the ground. testport_out reads 0, confirming the internal state register is in S0.

First Jump — JmpPow = 3 (approx. 10–160 ns): B_in goes high around 10 ns. On the next rising clock edge the processor transitions to the jumping state: Jmp_out goes high, Run_out goes low, and testport_out changes to 1. YPos_out steps through $0 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 5 \rightarrow 3$, tracing a symmetric parabolic arc as expected for an initial velocity of 3 with gravity of -1. When YPos_out returns to 0 at approximately 160 ns, the processor automatically transitions back to S0 — Run_out goes high, Jmp_out goes low, and testport_out returns to 0 — without any external signal, demonstrating correct closed-loop feedback.

Second Jump — JmpPow = 2 (approx. 200–280 ns): JmpPow_in changes to 2 and B_in is pulsed again. The processor initiates a shorter jump, with YPos_out stepping through $0 \rightarrow 2 \rightarrow 3 \rightarrow 2 \rightarrow 0$. The arc is noticeably shorter and faster than the first jump, correctly reflecting the lower initial velocity. The processor returns to running near 280 ns.

Third Jump — JmpPow = 2 (approx. 310–390 ns): A second jump with JmpPow = 2 repeats the same short arc: $0 \rightarrow 2 \rightarrow 3 \rightarrow 2 \rightarrow 0$, confirming consistent and repeatable **behavior**.

Fourth Jump — JmpPow = 6 (approx. 390–590 ns): JmpPow_in is set to 6, producing the longest jump in the simulation. YPos_out steps through $0 \rightarrow 6 \rightarrow 11 \rightarrow 15 \rightarrow 18 \rightarrow 20 \rightarrow 21 \rightarrow 20 \rightarrow 18 \rightarrow 15 \rightarrow 11 \rightarrow 6 \rightarrow 0$, peaking at 21 around 490 ns as expected (since the sum $6+5+4+3+2+1 = 21$). The processor correctly remains in the jumping state for the full duration and returns to running only when YPos_out reaches zero near 590 ns.

Extended Running Period (590 ns–600+ ns): After the fourth jump, B_in remains low and the processor holds steadily in the running state with Run_out high and YPos_out at 0 for the remainder of the simulation.

Overall: The processor simulation confirms correct full-system integration. The controller and datapath interact properly through all control and feedback signals, Run and Jmp are always complementary, and the character's position follows the correct physics for all three tested jump power values. The processor requires no external intervention to return to the running state — the Y_eq_zero feedback signal closes the loop automatically.

Conclusion

This project involved the design and VHDL implementation of a custom RTL processor for simulating the jump mechanic of an endless runner game. The design was approached using the standard custom processor methodology: behavior was first captured in a high-level state machine, which was then decomposed into a datapath handling arithmetic and storage and a controller implementing an FSM. The two components were connected in a top-level processor module and verified individually and together using Vivado simulation.

The processor worked correctly in all three testbenches. The character correctly initiates a jump in response to the button signal, follows a physically accurate parabolic trajectory driven by decreasing velocity, and automatically returns to the running state upon landing.

The most important lesson from this project was the value of carefully designing at the HLSM level before touching any VHDL code. Mistakes made at the HLSM stage — such as incorrect transition conditions or missing control signals — would have propagated into both the datapath and controller, making debugging significantly harder. Having the HLSM reviewed before implementation would save substantial time.

In future projects, one thing to do differently would be to more carefully verify the datapath schematic before beginning VHDL implementation. In particular, the choice of which signal to feed into the YPos adder (the mux output versus the register output) required careful reasoning about timing, and a more thorough schematic review step would make such decisions more deliberate from the start.

Appendix A- VHDL Code:

Datapath

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity jumper_datapath is
    Port (
        CLK          : in  STD_LOGIC;
        ld_YVel       : in  STD_LOGIC;
        clr_YVel      : in  STD_LOGIC;
        ld_YPos       : in  STD_LOGIC;
        clr_YPos      : in  STD_LOGIC;
        init_YVel     : in  STD_LOGIC;
        JmpPow        : in  STD_LOGIC_VECTOR(7 downto 0);
        Y_eq_zero     : out STD_LOGIC;
        Ypos          : out STD_LOGIC_VECTOR(7 downto 0);
        testport      : out STD_LOGIC_VECTOR(7 downto 0)
    );
end jumper_datapath;
```

architecture structural of jumper_datapath is

```
    component reg8
        Port ( CLK : in STD_LOGIC;
              ld  : in STD_LOGIC;
              clr : in STD_LOGIC;
              D   : in STD_LOGIC_VECTOR(7 downto 0);
              Q   : out STD_LOGIC_VECTOR(7 downto 0));
    end component;

    component mux8
        Port ( A : in STD_LOGIC_VECTOR(7 downto 0);
              B : in STD_LOGIC_VECTOR(7 downto 0);
              sel : in STD_LOGIC;
              Y : out STD_LOGIC_VECTOR(7 downto 0));
    end component;

    component adder8
        Port ( A : in STD_LOGIC_VECTOR(7 downto 0);
              B : in STD_LOGIC_VECTOR(7 downto 0);
              S : out STD_LOGIC_VECTOR(7 downto 0));
    end component;

    component eq8
        Port ( A : in STD_LOGIC_VECTOR(7 downto 0);
              B : in STD_LOGIC_VECTOR(7 downto 0);
              EQ : out STD_LOGIC);
    end component;

    signal YVelReg_Q : STD_LOGIC_VECTOR(7 downto 0);
    signal YPosReg_Q : STD_LOGIC_VECTOR(7 downto 0);
    signal YVel_dec : STD_LOGIC_VECTOR(7 downto 0);
    signal YVel_mux_out : STD_LOGIC_VECTOR(7 downto 0);
    signal YPos_sum : STD_LOGIC_VECTOR(7 downto 0);

begin

    U_YVel_adder : adder8
        port map (A => YVelReg_Q, B => x"FF", S => YVel_dec);

    U_YVel_mux : mux8
        port map (A => YVel_dec, B => JmpPow, sel => init_YVel, Y => YVel_mux_out);

    U_YVelReg : reg8
        port map (CLK => CLK, ld => ld_YVel, clr => clr_YVel, D => YVel_mux_out, Q => YVelReg_Q);

    U_YPos_adder : adder8
```

```

    port map (A => YPosReg_Q, B => YVel_mux_out, S => YPos_sum);

    U_YPosReg : reg8
    port map (CLK => CLK, ld => ld_YPos, clr => clr_YPos, D => YPos_sum,
Q => YPosReg_Q);

    U_eq : eq8
    port map (A => YPosReg_Q, B => x"00", EQ => Y_eq_zero);

    Ypos      <= YPosReg_Q;
    testport <= YVelReg_Q;

```

end structural;

Controller

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity jumper_controller is
    Port ( CLK      : in  STD_LOGIC;
          B        : in  STD_LOGIC;
          Y_eq_zero : in  STD_LOGIC;
          Run       : out STD_LOGIC;
          Jmp       : out STD_LOGIC;
          ld_YVel   : out STD_LOGIC;
          clr_YVel  : out STD_LOGIC;
          ld_YPos   : out STD_LOGIC;
          clr_YPos  : out STD_LOGIC;
          init_YVel : out STD_LOGIC;
          testport  : out STD_LOGIC_VECTOR(7 downto 0));
end jumper_controller;

```

architecture structural of jumper_controller is

```

    component dff
    Port ( CLK : in  STD_LOGIC;
          D   : in  STD_LOGIC;
          Q   : out STD_LOGIC);
    end component;

```

```

    component notgate
    Port ( A : in  STD_LOGIC;
          Y : out STD_LOGIC);
    end component;

```

```

    component andgate
    Port ( A : in  STD_LOGIC;
          B : in  STD_LOGIC;
          Y : out STD_LOGIC);

```

```

end component;

component orgate
    Port ( A : in  STD_LOGIC;
           B : in  STD_LOGIC;
           Y : out STD_LOGIC);
end component;

signal S, NS : STD_LOGIC;
signal S_not : STD_LOGIC;
signal Yeq_not: STD_LOGIC;
signal S_term1: STD_LOGIC;
signal S_term2: STD_LOGIC;

begin

    U_not_S    : notgate port map (A => S,      Y => S_not);
    U_not_Yeq  : notgate port map (A => Y_eq_zero, Y => Yeq_not);
    U_and1     : andgate  port map (A => S_not,   B => B,      Y => S_term1)
;
    U_and2     : andgate  port map (A => S,      B => Yeq_not, Y => S_term2)
;
    U_or       : orgate   port map (A => S_term1, B => S_term2, Y => NS);
    U_dff      : dff      port map (CLK => CLK,  D => NS,    Q => S);

    Run        <= not NS;
    Jmp        <= NS;
    ld_YVel    <= '1';
    clr_YVel   <= '0';
    ld_YPos    <= NS;
    clr_YPos   <= not NS;
    init_YVel  <= S_term1;
    testport   <= "0000000" & NS;

end structural;

```

Processor

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity jumper_processor is
    Port ( CLK      : in  STD_LOGIC;
           B        : in  STD_LOGIC;
           JmpPow   : in  STD_LOGIC_VECTOR(7 downto 0);
           Run      : out STD_LOGIC;
           Jmp      : out STD_LOGIC;
           YPos     : out STD_LOGIC_VECTOR(7 downto 0);
           testport : out std_logic_vector(7 downto 0));
end jumper_processor;

```

architecture structural of jumper_processor is

```
    component jumper_controller is
        Port ( CLK : in STD_LOGIC; B : in STD_LOGIC; Y_eq_zero : in STD_LOGIC
;
            Run : out STD_LOGIC; Jmp : out STD_LOGIC;
            ld_YVel : out STD_LOGIC; clr_YVel : out STD_LOGIC;
            ld_YPos : out STD_LOGIC; clr_YPos : out STD_LOGIC;
            init_YVel : out STD_LOGIC; testport : out std_logic_vector(7 d
ownto 0));
    end component;
```

```
    component jumper_datapath is
        Port ( CLK : in STD_LOGIC; ld_YVel : in STD_LOGIC; clr_YVel : in STD_
LOGIC;
            ld_YPos : in STD_LOGIC; clr_YPos : in STD_LOGIC;
            init_YVel : in STD_LOGIC; JmpPow : in std_logic_vector(7 downt
o 0);
            Y_eq_zero : out STD_LOGIC; Ypos : out std_logic_vector(7 downt
o 0);
            testport : out std_logic_vector(7 downto 0));
    end component;
```

```
    signal ld_YVel_wire    : STD_LOGIC;
    signal clr_YVel_wire   : STD_LOGIC;
    signal ld_YPos_wire    : STD_LOGIC;
    signal clr_YPos_wire   : STD_LOGIC;
    signal init_YVel_wire  : STD_LOGIC;
    signal Y_eq_zero_wire  : STD_LOGIC;
    signal ctrl_testport   : std_logic_vector(7 downto 0);
    signal dp_testport     : std_logic_vector(7 downto 0);
```

begin

```
    ctrl : jumper_controller
        port map (
            CLK => CLK, B => B, Y_eq_zero => Y_eq_zero_wire,
            Run => Run, Jmp => Jmp,
            ld_YVel => ld_YVel_wire, clr_YVel => clr_YVel_wire,
            ld_YPos => ld_YPos_wire, clr_YPos => clr_YPos_wire,
            init_YVel => init_YVel_wire, testport => ctrl_testport);
```

```
    dp : jumper_datapath
        port map (
            CLK => CLK,
            ld_YVel => ld_YVel_wire, clr_YVel => clr_YVel_wire,
            ld_YPos => ld_YPos_wire, clr_YPos => clr_YPos_wire,
            init_YVel => init_YVel_wire, JmpPow => JmpPow,
            Y_eq_zero => Y_eq_zero_wire, Ypos => Ypos,
```

```
        testport => dp_testport);  
    testport <= ctrl_testport;  
end structural;
```